BLACK PEN Answer all questions first. **GREEN PEN** Then: tick and rewrite mistakes.

LESSON 018 · Records, lists and tuples

Name: _____ Class: _____ Date: _____

Work through the pages in order. Answer in BLACK PEN first. Check in GREEN PEN only after the Exit Ticket.

Do Now · Recall previous learning

Q1. What does names[2] access in a 1D array?

Q2. In a C# 2D array organised as rows and columns, which dimension does GetLength(1) measure?

Q3. Which structure from last lesson has a fixed size once created?

Q4. Why should a student's name, candidate number and score be stored as separate related fields rather than one primitive value?

Do Now Recall previous learning | **I Do** Read new knowledge | **We Do** Practise with guidance | **You Do** Work independently | **Review & Improve** Check understanding

BLACK PEN Answer all questions first. **GREEN PEN** Then: tick and rewrite mistakes.

I Do · Read new knowledge

Key vocabulary

Recall the term — record

Recall the term — field

Recall the term — list

Recall the term — tuple

Recall the term — index

Recall the term — Add()

Recall the term — Remove()

New learning

Records group fields — CODE:
public record Student(string Name, int CandidateNumber, int Score);
Student pupil = new Student("Amelia", 1037, 72);
Console.WriteLine(pupil.Name); — A record represents one entity using several related fields. In C#, this positional record creates a named Student type with properties. Those properties use init, so use a with expression to create an updated copy.

Lists can change size — CODE:
List<string> names = new List<string>();
names.Add("Amelia");
names.Add("Ben");
names.Remove("Ben"); — A list is ordered like an array, but its size can change while the program runs.

Tuples group a few values — CODE:
var result = (Name: "Amelia", Score: 72);
Console.WriteLine(result.Name);
Console.WriteLine(result.Score); — A tuple groups values without declaring a reusable Student type. It is especially helpful when a method returns several results.

Choose by behaviour — CODE:
fixed size? → array
one item / fields? → record
grow or shrink? → list
small grouped set? → tuple — The exam may ask why one structure suits a context better than another.

BLACK PEN Answer all questions first. **GREEN PEN** Then: tick and rewrite mistakes.

Worked example

Build and use a C# record

1. Declare the record type — CODE:
public record Student(string Name, int CandidateNumber, int Score);

2. Create a student — CODE:
Student pupil = new Student("Amelia", 1037, 72);

3. Access a property — CODE:
Console.WriteLine(pupil.Score); // 72

4. Update using with — CODE:
Student updated = pupil with { Score = 80 };
Console.WriteLine(updated.Score); // 80

We Do · Practise with guidance

List operations

Complete code that adds Daisy then removes Ben.

Fill the same code

```
names.?("Daisy");
names.?("Ben");
```

```
names.Add("Daisy");
names.Remove("Ben");
```

Tuple access

A tuple stores a student's name and score.

```
var result = (Name: "Amelia", Score: 72);
Console.WriteLine(result.?);
Console.WriteLine(result.?);
```

```
var result = (Name: "Amelia", Score: 72);
Console.WriteLine(result.Name);
Console.WriteLine(result.Score);
```

Choose a structure

A playlist must grow and shrink while the program is running.

Complete the decision

```
Need ordered items
Need add/remove
```

```
Size changes
Best structure = ?

Need ordered items
Need add/remove
Size changes
Best structure = List
```

Declare a C# record

Define a Book record with Title, Author and ISBN fields, then create one book.

Complete the fields

```
Book record
Title = ?
Author = ?
ISBN = ?
```


Exit Ticket · Model answer

True. A C# List<T> can add and remove items as the program runs.

Try the question again

Choose one question you got wrong. All correct? Choose the hardest.

1. Copy the whole original question:

2. Write a complete new answer in GREEN PEN. Show working if needed:

```
{
public static void Main()
{
var result = (Name: "Amelia", Score: 72);
Console.WriteLine(result.Name);
Console.WriteLine(result.Score);
}
}
```

Check: Both tuple fields are accessed by name.

Q3. Find an item

CODE:

```
using System;
using System.Collections.Generic;

public class Program
{
public static void Main()
{
List<string> courses = new List<string> { "Maths", "Physics", "Computer Science",
"History", "Art" };
Console.Write("Course: ");
string wanted = Console.ReadLine();

if (courses.Contains(wanted))
Console.WriteLine("Found");
else
Console.WriteLine("Not found");
}
}
```

Check: A present and absent course both produce sensible output.

Q4. Explain the structure

CODE:

```
// Array: fixed-size ordered collection.
// List: ordered collection that can grow or shrink.
// Record: a reusable named type with related properties.
// Tuple: a small fixed group of related values.
```

Check: Each choice refers to the behaviour of the structure, not just its name.

You Do · Work independently

BLACK PEN: Write your own answers. Do not look at the Review & Improve pages yet.

Question 1

Create and edit a list

Create a List<string> of four names. Add a fifth, remove one name, then output every remaining item.

Question 2

Group a result

Create a tuple containing a student name and integer score. Output both named values.

Question 3

Find an item

Create a list of five course names. Ask for a course and report whether the list contains it.

Question 4

Choose and code a record

Write short comments explaining when you would choose an array, list, record-like structure and tuple.

BLACK PEN Answer all questions first. **GREEN PEN** Then: tick and rewrite mistakes.

Exit Ticket · Answer before checking

A C# List<T> can grow and shrink while the program is running.

RHS COMPUTING / OCR H446 / 1.4.2 LESSON 018 · DRAFT

BLACK PEN Answer all questions first. **GREEN PEN** Then: tick and rewrite mistakes.

Review & Improve · GREEN PEN

Now change to GREEN PEN. Find each original question. Tick correct answers; read the model and rewrite mistakes.

Page 1 onwards · Do Now

Q1. The third element. Array indexes start at 0.

Q2. The size of dimension 1 — the number of columns.

Q3. An array.

Q4. The data item has several related fields that belong together.

You Do pages · Independent questions

Q1. Create and edit a list
CODE:

```
using System;
using System.Collections.Generic;

public class Program
{
public static void Main()
{
List<string> names = new List<string> { "Amelia", "Ben", "Charlie", "Daisy" };
names.Add("Ethan");
names.Remove("Ben");

foreach (string name in names)
{
Console.WriteLine(name);
}
}
}
```

Check: The list changes size and every remaining item is displayed.

Q2. Group a result
CODE:

```
using System;

public class Program
```